



方法引用

北京理工大学计算机学院
金旭亮

引例

```
private static void useLambdaToPrint() {  
    //创建一个包容5个数字的集合  
    var nums :List<Integer> = List.of(1, 2, 3, 4, 5);  
    //使用Lambda遍历输出集合中的所有数字  
    nums.forEach(num -> System.out.println(num));  
}
```

- ▶ 上述示例使用的Lambda，只有一句，并且其含义非常明确，将一个整数作为参数传给println函数。
- ▶ 针对这种情况，编译器其实只需要知道要调用的函数是哪个，就可以推断出所有的其余信息。
- ▶ 为此，写代码时，其实只需要指明要调用的函数名即可，这就是“方法引用”特性引入的技术背景。

引入“方法引用”特性简化代码

```
private static void useMethodReferenceToPrint() {  
    var nums :List<Integer> = List.of(1, 2, 3, 4, 5);  
    //使用方法引用遍历输出集合中的所有数字  
    nums.forEach(System.out::println);  
}
```

只需指定方法名，简化了代码

示例：WhyUseMethodReference.java

“方法引用 (Method Reference)”



Java 8中，允许你把方法名字直接传给一个函数型接口变量，这实际上省去了定义Lambda表达式的这个步骤，其要求是：

Lambda表达式只有一句，并且是方法调用语句。



每个方法引用都存在一个等效的 `lambda` 表达式。

方法引用使用示例-1

```
class Person {  
    private String name;  
    private int age;  
  
    public Person(String name, int age) {...}  
  
    //getter and setter  
  
    @Override  
    public String toString() {  
        return name + " (" + age + ")";  
    }  
  
    public static int compareAge(Person p1, Person p2) {  
        Integer age1 = p1.getAge();  
        return age1.compareTo(p2.getAge());  
    }  
}
```

```
public static void main(String[] args) {  
    List<Person> people = new ArrayList<>();  
    //创建几个测试用Person对象, 加入到了people集合中  
    people.add(new Person(name: "张三", age: 48));  
    people.add(new Person(name: "李四", age: 30));  
    people.add(new Person(name: "王五", age: 24));  
}
```

MethodReference.java

示例中定义了一个Person类，这个类中又定义了一个静态方法compareAge()，一会儿我们会在代码中引用这个方法。

方法引用使用示例-2

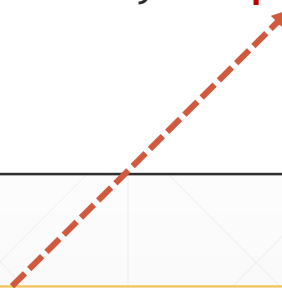
```
public class MethodReference {  
  
    public static void main(String[] args) {...}  
  
    public void sortDataWithInstanceMethod(List<Person> people) {...}  
  
    public void sortDataWithStaticMethod(List<Person> people) {...}  
  
    // 实例方法  
    public int compareAge(Person p1, Person p2) {  
        Integer age1 = p1.getAge();  
        return age1.compareTo(p2.getAge());  
    }  
}
```

在主程序中定义了一个compareAge()方法，这个方法后面也将被引用.....

我们要使用JDK中的sort方法排序

JDK的Collections类定义了一个sort方法，可以对一个对象集合进行排序.....

```
public class Collections {  
    .....  
    public static <T> void sort(List<T> list, Comparator<? super T> c) {  
        .....  
    }  
}
```

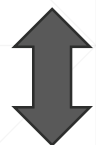


上述方法声明中的Comparator<T>是一个泛型的**函数型**接口（你可以查看一下JDK中此接口的声明，看看是不是有一个@FunctionalInterface注解），它能接收一个Lambda表达式作为实参.....

使用方法引用简化Lambda代码

```
public void sortDataWithInstanceMethod(List<Person> people) {  
    Collections.sort(people, (p1,p2)->this.compareAge(p1, p2));  
}
```

等价于



```
public void sortDataWithInstanceMethod(List<Person> people) {  
    // 实例方法引用  
    Collections.sort(people, this::compareAge);  
}
```

只有一句，并且是对特定方法的直接调用.....



上述代码把MethodReference类所定义的compareAge方法当作实参直接传给Collections.sort方法。

同样支持静态方法引用

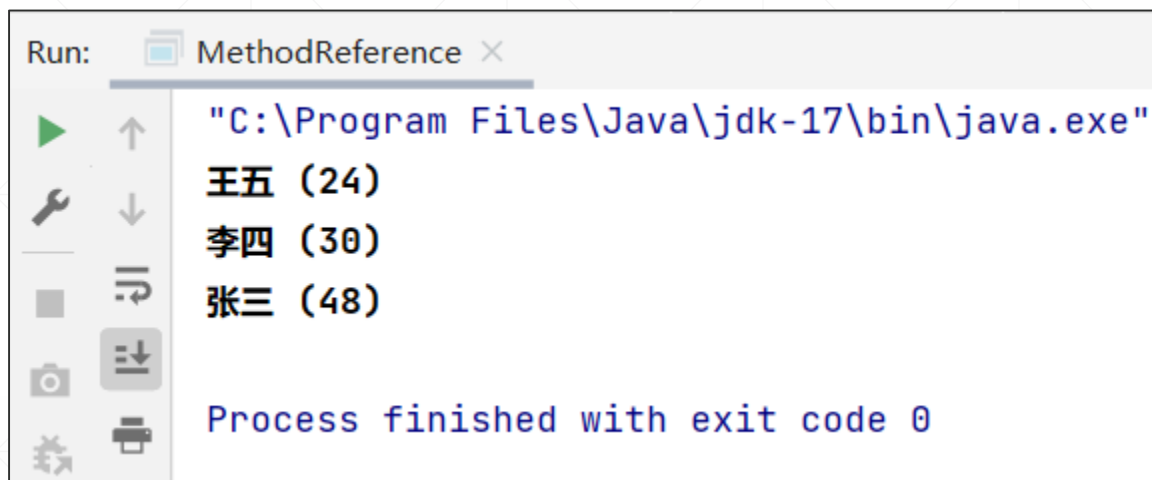
Java 8的方法引用特性，同样支持静态方法，以下代码把Person类所定义的静态方法compareAge作为实参传给Collections.sort方法：

```
public void sortDataWithStaticMethod(List<Person> people) {  
    // 静态方法引用  
    Collections.sort(people, Person::compareAge);  
}
```



Java 8的方法引用特性，实际上是Lambda表达式的一种简写，它进一步地精简了代码，使代码更加易读。

示例运行截图



通过向`Collections.sort`方法传送排序函数，实现了对`Person`集合中对象的排序。

更多的例子

JDK中可以使用Timer组件实现定时调用，Timer组件构造方法的第二个参数，就可以传入一个方法引用：

```
static void useMethodReference() {  
    //以下两句是等价的  
    // (1) 使用Lambda表达式构造Timer对象  
    //var timer=new Timer(1000, event-> System.out.println(event));  
    // (2) 使用方法引用构造Timer对象  
    var timer = new Timer( delay: 1000, System.out::println);  
    timer.start();  
    System.out.println("敲回车键退出.....");  
    new Scanner(System.in).nextLine();  
}
```

引用类的“默认构造函数”

```
public class UseSupplier {  
    public static void main(String[] args) {  
        Supplier<ExampleClass> supplier = () -> new ExampleClass();  
        System.out.println(supplier.get());  
        //使用方法引用达到同样的目的  
        Supplier<ExampleClass> supplier2 = ExampleClass::new;  
        System.out.println(supplier2.get());  
    }  
}  
  
class ExampleClass {  
}
```

Supplier<T>是JDK所定义的一个函数式接口，它接收一个无参函数，必须返回一个类型T的实例。



如果一个类定义了多个构造函数，则Java会依据上下文来推断使用哪个。

引用类的“有参构造函数”

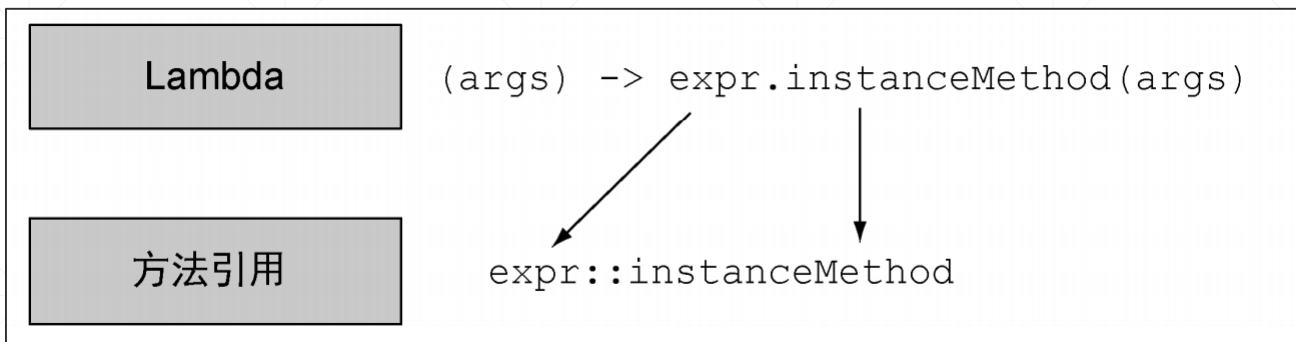
```
public class ReferenceConstructor {  
    public static void main(String[] args) {  
        Function<Integer, SomeClass> creator = SomeClass::new;  
        System.out.println(creator.apply(t: 100).getValue());  
    }  
}  
  
class SomeClass {  
    private int value;  
  
    public SomeClass(int value) {  
        this.value = value;  
    }  
  
    public int getValue() { return value; }  
}
```

Function接口指定方法有一个Integer参数，它可以被转换int，所以，这里Java会调用SomeClass的有参构造函数。

调用apply()方法，
传入实参

此类的构造函数，需要接收一个int类型的参数。

方法引用语法特性总结-1



//原始方式

```
Stream.of(3, 1, 4, 1, 5, 9)
    .forEach(x -> System.out.println(x));
```

//方式一: object::instanceMethod

```
Stream.of(3, 1, 4, 1, 5, 9)
    .forEach(System.out::println);
```

ThreeTypeMethodReference.java

方法引用语法特性总结-2

Lambda

(args) -> ClassName.staticMethod(args)

方法引用

ClassName::staticMethod



```
Stream.generate(() -> Math.random())  
    .limit(10)  
    .forEach(System.out::println);  
  
//方式二: Class::staticMethod  
Stream.generate(Math::random)  
    .limit(10)  
    .forEach(System.out::println);
```

方法引用语法特性总结-3

Lambda

`(arg0, rest) -> arg0.instanceMethod(rest)`

arg0是ClassName
类型的

方法引用

`ClassName::instanceMethod`

```
List<String> strings =  
    Arrays.asList("this", "is", "a", "list", "of", "strings");  
//原始方式  
strings.stream()  
    .sorted((s1, s2) -> s1.compareTo(s2))  
    .forEach(System.out::println);  
//方式三: Class::instanceMethod  
//第一个参数作为对象引用, 第二个 (第三参数...) 作为方法参数  
strings.stream()  
    .sorted(String::compareTo)  
    .forEach(System.out::println);
```


小技巧



```
public class IntelliJCanHelp {  
    public static void main(String[] args) {  
        //将鼠标移到高亮显示的部分，可以看到IntelliJ给的提示  
        BinaryOperator<Integer> sum = (x, y) -> x + y;  
        //输出: 300  
        System.out.println(sum.apply(10, 20));  
    }  
}
```

Lambda can be replaced with method reference

Replace lambda with method reference Alt+Shift+Enter

采纳IntelliJ给的提示，相应代码会自动地简化为使用“方法引用”，所以，方法引用这个语法特性，其实是不需要死记语法规则的。

结论：方法引用是块“语法糖”



如果一个 Lambda 代表的只是“直接调用这个方法”，那最好还是用名称来调用它，而不是去描述如何调用它。事实上，方法引用就是让你根据已有的方法实现来创建Lambda表达式。



方法引用是一种取代编写Lambda表达式的快捷方法。



你可以把方法引用看作针对仅仅涉及单一方法的Lambda的“语法糖”，因为你实现同样的功能时，要写的代码变少了。

本讲小结



本讲介绍了Java 8为简化Lambda表达式所引入的“方法引用”特性，此特性是块“语法糖”，它并没有增强程序的功能，只是让我们写的代码数量更少。



不使用方法引用，也能完成同样的编程任务，尽管如此，还是应该掌握方法引用的使用方法，至少能看得懂。



方法引用，在使用“函数式编程”范式的Java代码中，还是比较常用的。

巩固练习



请你自己随意设计几个函数式接口（有参数和无参数的），然后，编写几个静态方法或实例方法，这些方法能满足函数式接口中的方法签名，之后，编写示例代码，试着使用本讲介绍的“方法引用”特性编程，以便熟悉这一特性。